MLOps Exam Assignment

Weather Rain Prediction MLOps Pipeline

Group Name

Amisha Magar

Samad Ahmad

Nasrin Miya

1. Introduction

This project develops a complete MLOps pipeline to predict whether it will rain tomorrow based on weather data.

The main objective is to build a reproducible and deployable machine learning system rather than focusing only on model performance.

The system includes data ingestion, preprocessing, model training, deployment via API, monitoring, and a user-facing dashboard.

2. Pipeline Overview

The pipeline consists of the following components:

- **Data Ingestion:** Weather data is collected from the Open-Meteo API.
- **Data Preprocessing:** Raw data is cleaned and prepared for training.
- **Feature Engineering:** Relevant features are selected for prediction.
- **Model Training:** A machine learning model is trained on processed data.
- **Deployment:** The trained model is deployed using FastAPI.
- **Monitoring:** Logs and metrics are used to track system performance.

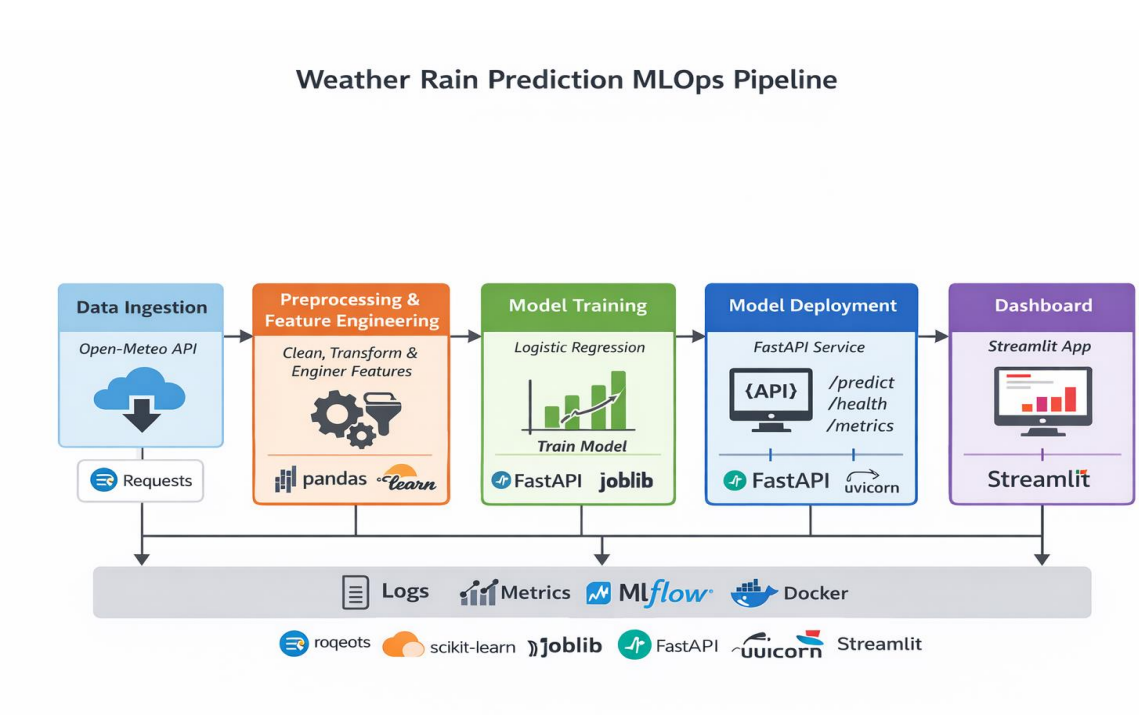

Figure: End-to-end MLOps pipeline showing data ingestion, preprocessing, model training, deployment, and monitoring.

3. Data Ingestion

Weather data is collected using the Open-Meteo API.

The data includes:

- Maximum temperature
- Minimum temperature
- Precipitation

The raw data is stored in:

data/raw/weather.csv

This step ensures that the system can continuously collect new data for future predictions.

4. Data Preprocessing

The raw data is cleaned and prepared for model training.

Key preprocessing steps include:

- Creating the target variable (rain_tomorrow) using shifted precipitation data
- Converting the target into binary classification (1 = rain, 0 = no rain)
- Removing missing values created during shifting
- Selecting relevant features for training

The processed dataset is stored in:

data/processed/weather_processed.csv

5. Feature Engineering

Feature engineering is applied to improve model performance.

The model uses the following features:

- Temperature (max and min)

- Precipitation

- Temperature range

- Lag-based precipitation (previous day)

These derived features help the model capture additional patterns in the data and improve prediction quality.

6. Model Training

A Logistic Regression model is used for binary classification.

The dataset is split into training and testing sets using an 80/20 ratio.

The model is evaluated using accuracy, precision, recall, and F1-score.

The trained model is saved as:

models/rain_model_v1.pkl

Model Performance

- Accuracy: 0.56
- Precision: 0.43
- Recall: 0.43
- F1-score: 0.43

The model achieves moderate performance. The results indicate that the model can capture some patterns in the data, but performance is limited due to the small dataset and simple features.

However, the focus of this project is on building a complete MLOps pipeline rather than achieving perfect model accuracy.

7. Deployment (API)

The trained model is deployed using FastAPI.

The system provides the following endpoints:

- / → Home endpoint
- /health → Health check
- /predict → Prediction endpoint
- /metrics → Monitoring and performance metrics

The API accepts JSON input and returns prediction results in real time.

The system is designed to be triggered on demand via API requests, enabling real-time predictions.

Example flow:

User input → API → Model → Prediction → API response

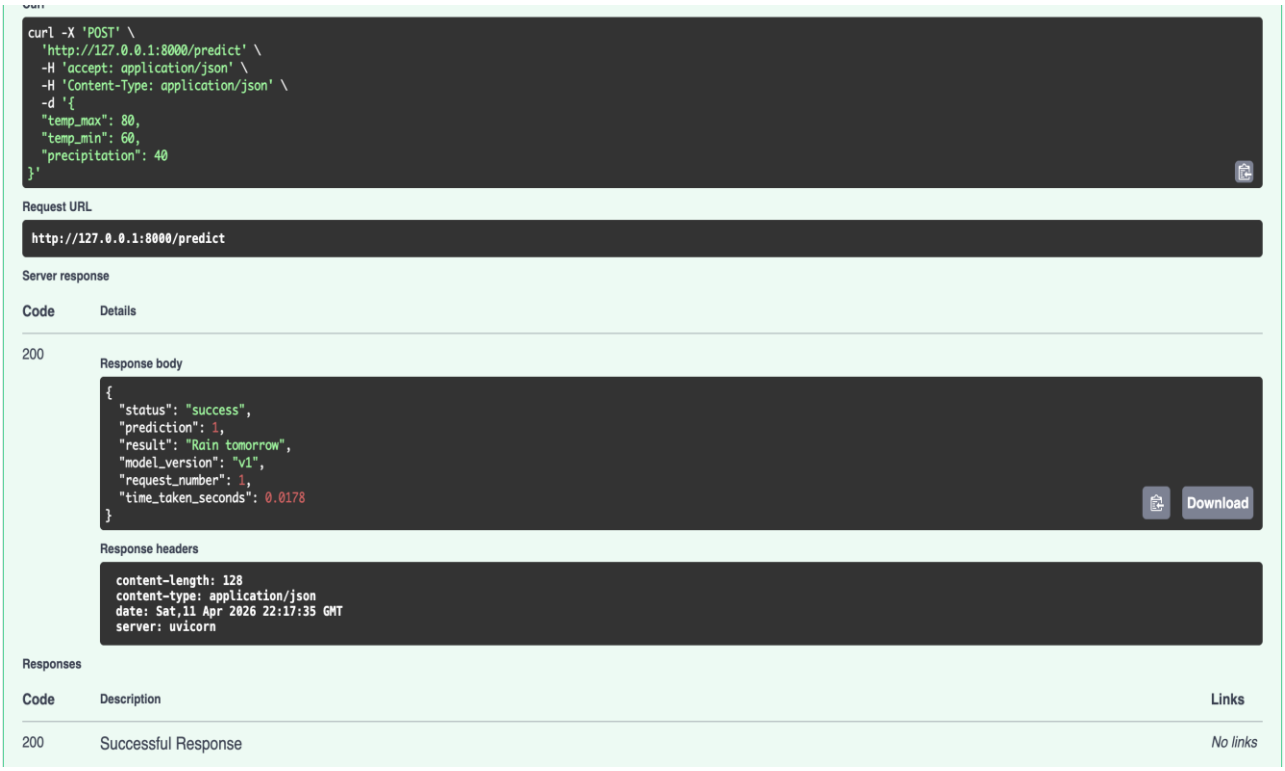

Figure 2: FastAPI Swagger UI showing prediction request and response.

8. Monitoring & Logging

The system includes basic monitoring to track performance and usage.

Monitoring features:

- Request count
- Success and error tracking
- Prediction time tracking
- Model version tracking

Logs are stored in:

logs.log

Metrics are stored in:

metrics.json

This ensures the system is debuggable, traceable, and allows monitoring of system performance over time.

9. Experiment Tracking (MLflow)

MLflow is used to track experiments, parameters, and model performance.

Tracked components include:

- Model parameters
- Evaluation metrics
- Model artifacts

The tracking data is stored in:

mlruns/

This allows comparison of different training runs and improves reproducibility.

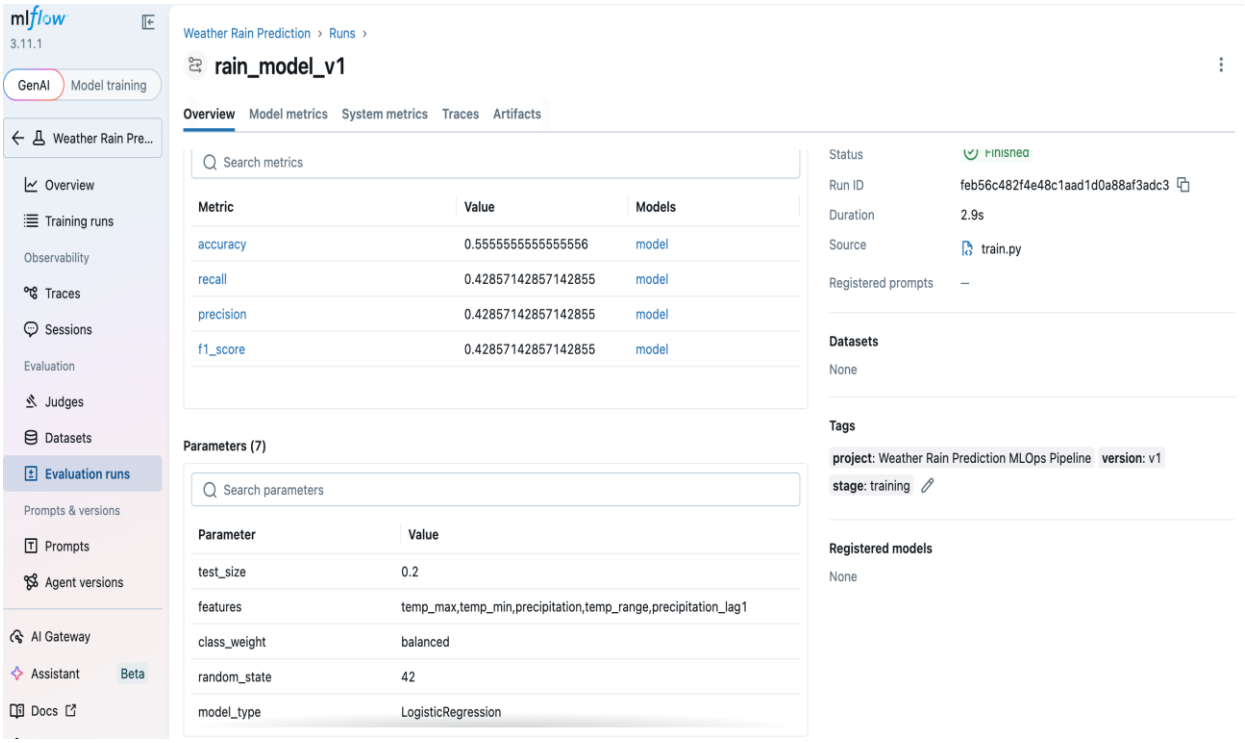

Figure 3: MLflow interface showing experiment tracking, parameters, and model metrics.

10. Dashboard (Streamlit)

A Streamlit dashboard is developed to provide a user-friendly interface.

The dashboard allows users to:

- Input weather values
- Get predictions instantly
- View model performance metrics

This makes the system accessible to non-technical users.

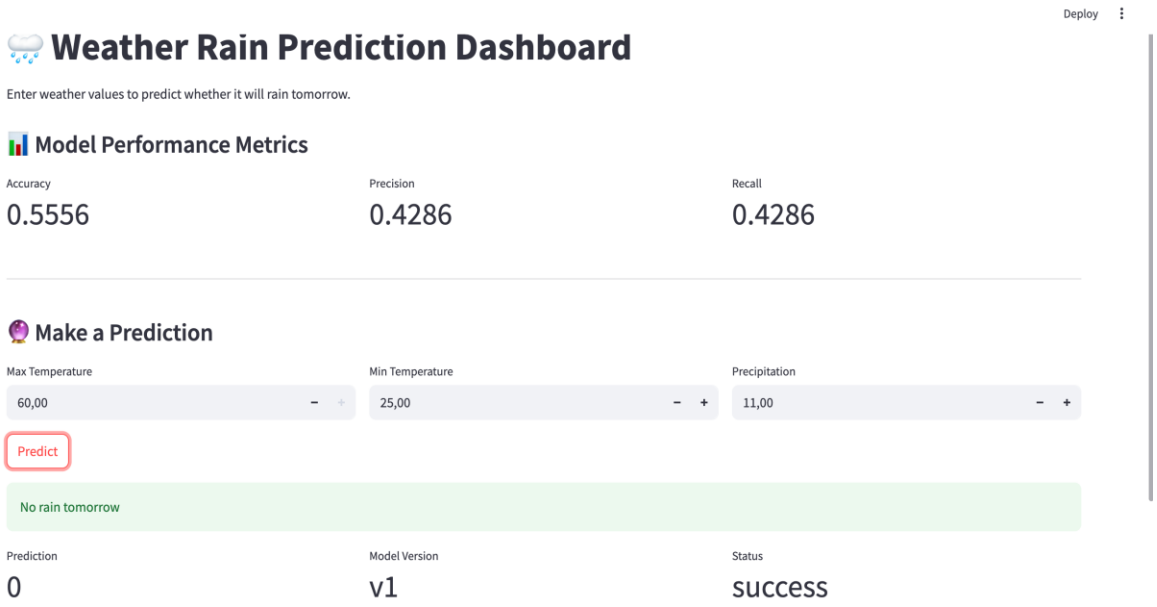

Figure 4: Streamlit dashboard showing user input, prediction results, and model performance metrics.

11. Containerization (Docker)

The project is containerized using Docker.

Docker ensures:

- Reproducibility
- Consistent environment
- Easy deployment

The application can be run in any system without dependency issues.

12. Limitations

One limitation of this system is the relatively small dataset and simple model, which affects prediction performance.

The model may not generalize well to all weather conditions.

Future improvements could include:

- Using larger datasets
- Adding more features
- Applying more advanced machine learning models

13. Conclusion

This project successfully demonstrates a complete MLOps pipeline from data ingestion to deployment and monitoring.

Although the model performance is moderate, the system effectively shows how machine learning models can be operationalized in real-world environments.

The project highlights key MLOps concepts such as reproducibility, deployment, monitoring, and experiment tracking, demonstrating a practical production-ready system.